



CryptoStream

Real-time multi-exchange crypto pipeline — unifying Coinbase, Binance, and Kraken into a single reproducible, production-grade workflow.

Michael Hein · 300375535 · CSIS 4495 — Applied Research Project ·
Section S3

[GITHUB.COM/MICHAELHEIN915/W26_4495_S3_MICHAELH](https://github.com/MICHAELHEIN915/W26_4495_S3_MICHAELH)

The Problem

One Pipeline. Three Exchanges. Zero Spreadsheets.



Utility

Operators and researchers today juggle **three separate exchange tabs** alongside manual spreadsheets to monitor live prices, volatility, and alerts — a fragmented, error-prone workflow.

Novelty

CryptoStream normalizes **Coinbase + Binance + Kraken** into a **single Kafka topic**, pairing a live dashboard with Slack/email alerts, Isolation Forest anomaly detection, optional S3/Athena SQL, Prometheus/Grafana observability, and optional local LLM + sentiment analysis — all in one **Compose-centered workflow**.

Project Goals at a Glance



Unified Stream

All venues normalized into one Kafka topic with a shared schema



Real-Time Dashboard

Flask + Chart.js UI with KPIs, volatility, and cross-exchange comparison



Alerts + ML

Slack/email rules and Isolation Forest on rolling trade features



Optional SQL/BI

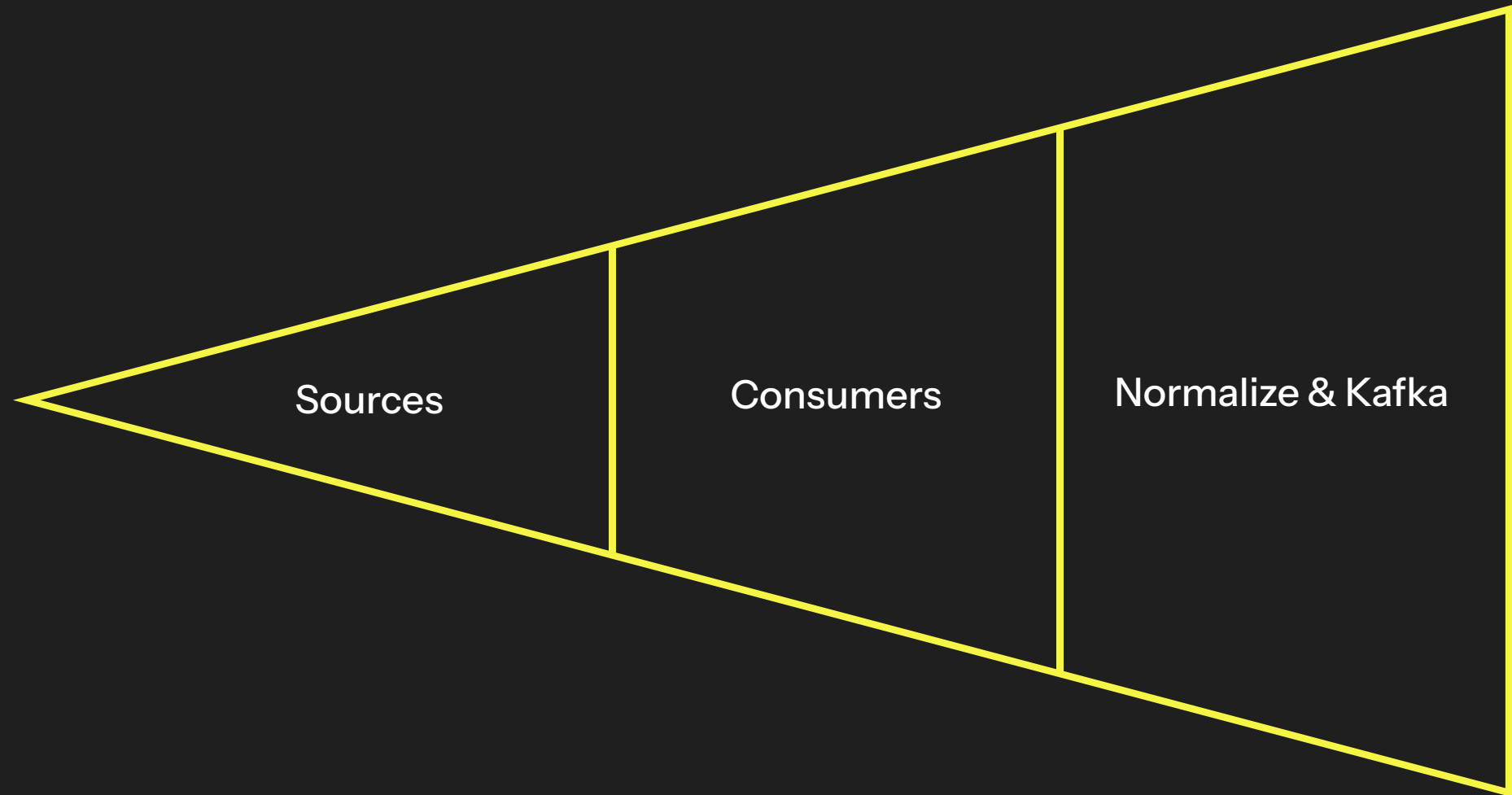
S3 Parquet → Athena queries and QuickSight/Redshift documented paths



Optional LLM + Sentiment

Local MLX + news RSS → VADER sentiment integrated via live dashboard JSON

System Architecture



All exchange feeds share a common parser — symbol, price, size, timestamp, exchange — before entering **crypto.trades.raw**. Kafka decouples ingestion from every downstream consumer, enabling independent scaling and fault isolation.

Built on Battle-Tested Open Standards



Orchestration

Docker Compose · Kafka
(Confluent) · Python 3.11

Processing & API

PySpark 3.5 (minute windows,
checkpoints) · Flask + Chart.js

Observability

Prometheus scraping · Grafana
Crypto Pipeline dashboard

Optional Cloud

AWS S3, Athena, QuickSight,
Redshift

Core Implementation

1

Producers

multi_exchange_producer.py — shared parse emits symbol, price, size, time, exchange per trade

2

Kafka Broker

Decouples all producers from PySpark and the Flask API; buffers bursts gracefully

3

Flask UI

Chart.js KPIs, live trades, volatility, volume, **cross-exchange spread**, and watchlist views

4

Stream Ops

Slack/email alerts on spread & volume rules + **Isolation Forest** on rolling feature windows



Optional Extensions

Modular Paths Beyond the Core



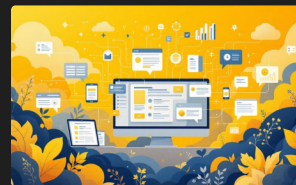
Cloud (S3 / Athena)

Parquet sink to S3, Athena DDL scripts, QuickSight helpers, and Redshift migration documented.



Observability

Prometheus scrapes pipeline metrics; Grafana *Crypto Pipeline* dashboard visualizes throughput and lag under multi-symbol load.



News + Sentiment

RSS / CryptoPanic → Kafka → VADER scoring surfaced at **/news** endpoint and integrated APIs.



Local LLM (MLX)

MLX on host, OpenAI-compatible API; live dashboard JSON injected as context via **host.docker.internal**.

Evaluation, Challenges & Lessons

1

Testing

pytest + GitHub Actions
CI; full Compose end-to-end: trades → UI → /health

2

Load Validation

Grafana under multi-symbol load; qualitative review of anomaly flags and LLM outputs

3

Key Fixes

WebSocket/timezone drift → shared config & parsers ·
Spark checkpoints & Docker RAM · AWS region/bucket alignment · MLX via extra_hosts

📄 Lessons Learned

- **Centralize configuration** — a single source of truth eliminates drift across producers, Spark, and Flask
- **Separate analytics from HTTP** — keeps Spark jobs testable independent of the API layer
- **Observability first** — instrument early so Grafana surfaces bottlenecks before they become incidents

Live Demo

Follow the Trade End-to-End



Producer Logs

Dashboard UI

Alert / Anomaly

The demo traces a single trade from raw WebSocket event through the Kafka topic into the Flask UI, triggering either a **spread/volume alert** or an **Isolation Forest anomaly flag**. Optional extensions — Grafana, Athena, and the MLX AI layer — available on request.



Takeaway

Real-time systems are integration.

Schema, consumers, monitoring, and UX must be designed **together** — not bolted on after the fact. CryptoStream demonstrates that a single, reproducible Compose workflow can unify three live exchanges, full observability, ML-based anomaly detection, and optional cloud SQL into one coherent, operational system.

Repository

github.com/MichaelHein915/W26_4495_S3_MichaelH · branch **main**

Questions?

Michael Hein · 300375535 · CSIS 4495 Section S3